

Homework 2

I. Introduction

In this assignment, you will use the MIPS simulator: **Mars** to write programs with a focus on basic MIPS instructions. The goal is to help you become familiar with the core operations of the MIPS instruction set. This assignment will primarily use the following MIPS instructions:

Arithmetic and Logical Instructions:

- *add, sub*: Perform addition and subtraction.
- *addi*: Add an immediate value to a register.
- *and, or, not*: Perform bitwise operations: AND, OR, and NOT.

Memory Access Instructions:

- *lw, sw*: Load and store data from and to memory.

Shift Instructions:

- *sll, srl*: Shift bits to the left or right.

Conditional Branching and Jump Instructions:

- *beq, bne*: Branch based on equality or inequality.
- *slt, slti*: Set a register to 1 if one value is less than another.
- *j*: Unconditional jump.

There are additional instructions not covered in the course that you need to use for printing messages or scanning user input:

Print/Scan Instructions

- *la (load address)*: Loads the address of a memory location or label into a register.
- *Syscall*: Triggers a system call, allowing the program to request specific services (e.g., input/output) from the operating system based on the value in register *\$v0*, for more system call functions, please refer to the **IV. Appendix**.

Example.

```
.data
prompt: .asciiz "Please enter a number: " # Define a prompt message for input
result_msg: .asciiz "The number as unsigned integer: " # Message for the result
newline: .asciiz "\n" # Newline character for formatting

.text
```

```

.globl main
main:
    # Print the prompt message to request input
    la    $a0, prompt      # Load the address of the string 'prompt' into $a0
    addi $v0, $zero, 4      # Set $v0 to 4 (print_string) to print the prompt
    syscall                # Execute the syscall to print the prompt

    # Read an integer from the user
    addi $v0, $zero, 5      # Set $v0 to 5 (read_int) using addi
    syscall                # Execute the syscall to read the integer input
    add $t0, $v0, $zero     # Store the user's input from $v0 into $t0

    # Print the result message
    la    $a0, result_msg   # Load the address of the result message
    addi $v0, $zero, 4      # Set $v0 to 4 (print_string)
    syscall                # Print the result message

    # Print the number as a signed integer
    add $a0, $t0, $zero     # Move the number to $a0 for printing
    addi $v0, $zero, 1      # Set $v0 to 1 (print signed int)
    syscall                # Print the signed integer

    # Print a newline for formatting
    la    $a0, newline      # Load address of newline string
    addi $v0, $zero, 4      # Set $v0 to 4 (print_string)
    syscall                # Print the newline

    # Exit the program
    addi $v0, $zero, 10     # Set $v0 to 10 (exit) using addi
    syscall                # Execute the syscall to exit the program

```

Multiplication is another instruction not covered in the course but will be needed for this assignment. We will discuss the implementation details of multiplication in future lessons. For now, you can use it just like addition and subtraction instructions.

Arithmetic Instructions

- *mul* (*multiply*): Perform multiplication.

Example.

```

# Perform the multiplication
mul $t2, $t0, $t1          # Multiply $t0 and $t1, store the result in $t2

```

II. Implement: Binary Exponentiation

1. Overview

Binary exponentiation is a method for efficiently computing large powers. Instead of multiplying a number repeatedly, the algorithm reduces the number of multiplications by using the binary form of

the exponent. It is widely used in areas like cryptography (e.g., RSA encryption) and numerical computations.

2. Core Concept

The **mathematical foundation of binary exponentiation** lies in the fact that: Even exponents can be broken down into two smaller exponents, which significantly reduces the size of the problem.

- If the exponent n is even:

$$x^n = (x^{\frac{n}{2}})^2 = (x^{\frac{n}{2}}) \cdot (x^{\frac{n}{2}})$$

- If the exponent n is odd:

$$x^n = x \cdot (x^{\frac{n-1}{2}})^2 = x \cdot (x^{\frac{n-1}{2}}) \cdot (x^{\frac{n-1}{2}})$$

The **binary representation of the exponent** optimizes the process on the computational level:

- Using the binary form of the exponent helps identify which bits require multiplication (when the bit is 1) and which can be skipped (when the bit is 0), improving efficiency.
- Through **bitwise operations** and **right shifts**, the exponentiation can be performed in logarithmic time, making it much faster than repeated multiplication.

Example: Calculate 5^{13}

$$\begin{aligned} \text{Since } 13_{10} &= 2^3 + 2^2 + 2^0 = 1101_2, \\ \text{then } 5^{13} &= 1 * 5^{2^3} + 1 * 5^{2^2} + 0 * 5^{2^1} + 1 * 5^{2^0} = 5^{2^3} \cdot 5^{2^2} \cdot 5^{2^0} = 625^2 \cdot 25^2 \cdot 5 \end{aligned}$$

3. Basic Algorithm

The following is the basic process and a Python implementation for reference.

```
Initialize: Result = 1, base = a
For each bit in the binary exponent:
    1. If the bit is 1, Result = Result * base
    2. base = base * base
Repeat until all bits are processed.
```

Python implementation¹:

```
def binary_power(base, exponent):
    result = 1
    while exponent > 0:
        if exponent & 1:
            result = result * base
        base = base * base
        exponent >>= 1
    return result
```

¹ <https://oi-wiki.org/math/binary-exponentiation/>

Please implement the Binary Exponentiation algorithm using the MIPS instructions discussed in Section I.

4. I/O Definition for Binary Exponentiation:

Input:

- base: The base a , the number to be exponentiated (a **signed integer**, $-2^{31} \leq a < 2^{31}$).
- exponent: The exponent n , the power (a **signed integer**, $-2^{31} \leq n < 2^{31}$).

Output:

- result: The final result a^n , the computed value of the exponentiation (a **signed integer**, overflow handling is not required, as this is expected in specific application scenarios.)
- hamming weight of the result: the total count of '1's in binary representation of the result. For example, the hamming weight of 6 is 2, because its binary representation is 110, which contains two '1's.

Requirement:

- If the **exponent** is negative, display the error message:

```
Error: Exponent cannot be negative.
```

- If both the **base** and **exponent** are zero, terminate the program and display:

```
Both base and exponent are zero. Exiting.
```

Example:

```
base: 2
exponent: 5
result: 32
hamming weight: 1

base: -2147483648
exponent: 2147483647
result: 0
hamming weight: 0

base: 3
exponent: 1000000001
result: -1943431165
hamming weight: 11

base: -2
exponent: 3
result: -8
hamming weight: 29

base: 2
exponent: -5
Error: Exponent cannot be negative.

base: 0
exponent: 0
Both base and exponent are zero. Exiting.
```

5. Hint

- You can directly translate the C++ code in the following to MIPS: (Don't use NLP model to generate your Homework code)

```
#include <iostream>

int main() {
    int base, exponent;

    while (true) {
        // --- 1. Read Inputs ---
        std::cout << "base: ";
        std::cin >> base;

        std::cout << "exponent: ";
        std::cin >> exponent;

        // --- 2. Handle Edge Cases ---
        if (base == 0 && exponent == 0) {
            std::cout << "Both base and exponent are zero. Exiting." <<
std::endl;
            break;
        }

        if (exponent < 0) {
            std::cout << "Error: Exponent cannot be negative.\n\n";
            continue;
        }

        // --- 3. Binary Exponentiation Algorithm ---
        int power_result = 1;
        int temp_base = base;
        int temp_exponent = exponent;

        while (temp_exponent > 0) {
            if (temp_exponent & 1) {
                power_result *= temp_base;
            }
            temp_base *= temp_base;
            temp_exponent >>= 1;
        }

        // --- 4. Print the Power Calculation Result ---
        std::cout << "result: " << power_result << std::endl;

        // --- 5. Hamming Weight Calculation (using a for loop) ---
        int weight_count = 0;
        for (int i = 0; i < 32; ++i) {
            if ((power_result >> i) & 1) {
                weight_count++;
            }
        }

        // --- 6. Print the Hamming Weight Result ---
        std::cout << "hamming weight: " << weight_count << std::endl;
        std::cout << std::endl;
    }
    return 0;
}
```

6. Submission

Please name your assembly program using your student ID. For example **arch_hw2_109xxx.asm**, and upload the file to eeclass.

7. Grading Criteria

- Correctness of basic algorithm: 60% = 20% + 20% + 20%
 - During the evaluation, the simulator's run speed is set to **30 inst/sec**, with a time limit of **5 minutes** for each testcase.
 - 3 random testcase, each 20% (result: 10%, hamming weight: 10%).
- Correctness of requirement: 20% = 10% + 10%
 - Negative exponent condition , 1 random testcase.
 - Exiting condition
- Message format: 20% = 10% + 10%
 - Input/Output messages format

III. Platform: MARS(MIPS Assembler and Runtime Simulator)

1. MARS can assemble and simulate the execution of MIPS assembly language programs. Please refer to the following URL to download Mars4_5.jar:

<https://github.com/dpetersanderson/MARS/releases/tag/v.4.5.1>

MARS v.4.5 Latest

dpetersanderson released this Oct 7, 2024 · 24 commits to main since this release · v.4.5.1 · a2f0a82

The last official MARS (original) release. August 2014. This is an executable JAR (Java ARchive) file. You can download it and execute directly.

Currently, the source code ZIP and TAR files shown here contain no source code. Github created those files automatically. All source code is contained within the JAR file.

▼ Assets 3

Mars4_5.jar	3.98 MB	Oct 7, 2024
Source code (zip)		Oct 7, 2024
Source code (tar.gz)		Oct 7, 2024

21 8 28 people reacted

2. MARS is developed with Java language, and it requires JRE installed on your computer. Please refer to the following URL to download JDK:

<https://www.oracle.com/java/technologies/downloads/?er=221886>

Java SE Development Kit 24.0.2 downloads

JDK 24 binaries are free to use in production and free to redistribute, at no cost, under the [Oracle No-Fee Terms and Conditions](#) (NFTC).

JDK 24 will receive updates under these terms, until September 2025, when it will be superseded by JDK 25.

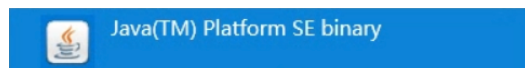
Linux macOS **Windows**

Product/file description	File size	Download
x64 Compressed Archive	229.62 MB	https://download.oracle.com/java/24/latest/jdk-24_windows-x64_bin.zip (sha256)
x64 Installer	205.86 MB	https://download.oracle.com/java/24/latest/jdk-24_windows-x64_bin.exe (sha256)
x64 MSI Installer	204.61 MB	https://download.oracle.com/java/24/latest/jdk-24_windows-x64_bin.msi (sha256)

3. After downloading MARS, it will be a “jar” file. Please DO NOT decompress it. Once you have successfully installed the JDK, you can open the file directly.



Mars4_4.jar



(Windows)

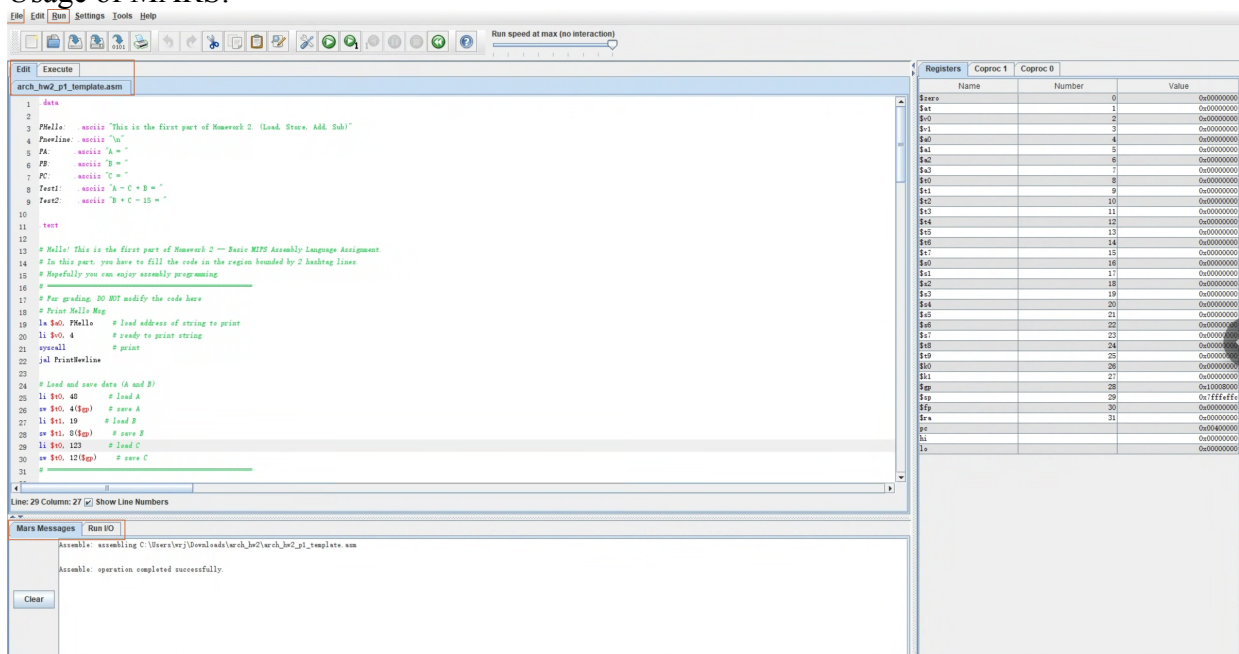


Mars4_5.jar

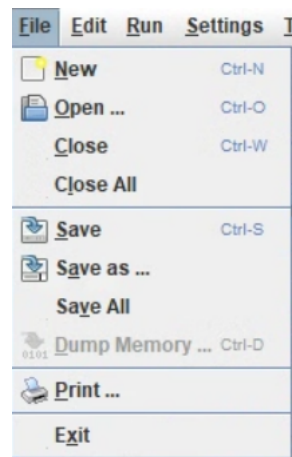


(Mac)

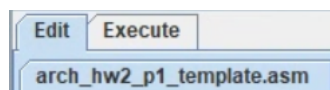
4. Usage of MARS:



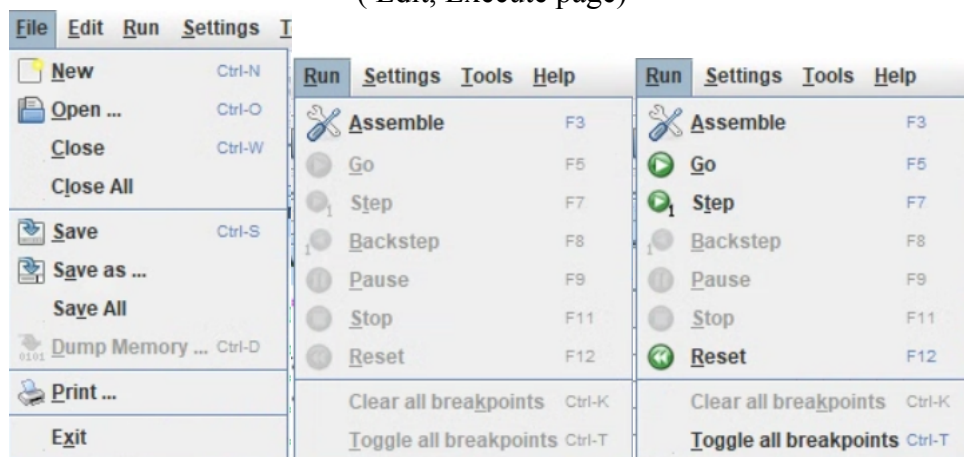
(Main Interface)



(Open, New and Save)



(Edit, Execute page)



(To run the code: Save -> Assemble -> Go)



(Program interaction interface)

IV. Appendix

MIPS system calls: <https://student.cs.uwaterloo.ca/~isg/res/mips/traps>)